Document number	P2467R0
Date	2021-11-15
Audience	LEWG
Reply-to	Jonathan Wakely <cxx@kayari.org>

Support exclusive mode for fstreams

-
  - Introduction
  - Background
  - Design Considerations
  - Proposed wording

Introduction

Historically, C++ iostreams libraries had a `noreplace` open mode that corresponded to the `O_EXCL` flag for POSIX `open`. That mode was not included in the C++98 standard, presumably for portability reasons, because it wasn't in ISO C90.

Since then, ISO C added support for "exclusive" mode to `fopen`, so now C++'s `<fstream>` is missing a feature that is present in both ISO C and POSIX. We should fix this for C++23.

Background

C11 added an 'x' modifier to the `fopen` flags for files opened in write mode. This opens the file in "exclusive" mode, meaning the `fopen` call fails if the file already exists. This is quite an important feature for certain use cases. As the WG14 N1339 proposal explained, "This is necessary to eliminate a time-of-creation to time-of-use race condition vulnerability. [...] fopen() does not indicate if an existing file has been opened for writing or a new file has been created. This may lead to a program overwriting or accessing an unintended file." See N1339 for additional rationale.

C++ already incorporates the C11 changes to `fopen` by reference, but `std::fstream` has no way to achieve the same thing. To avoid the time-of-creation to time-of-use (TOCTTOU) problem it's necessary to use `fopen` and `FILE*` (or non-standard APIs to hand an existing file handle or file descriptor to an fstream).

The 'x' modifier is widely supported. It was already supported as an extension by Glibc's `fopen` (since at least version 2.4 from 2006), and is in the draft for the next revision of POSIX (because it's rebasing on C11).

Support for opening an ofstream in exclusive mode isn't even a new idea, pre-ISO iostreams provided it. References to a `noreplace` flag can be found in texts such as:

- Object Oriented Programming with C++, 2nd Edition (presumably not corrected from the 1st edition published in 1999),
- Migration from UNIX System Laboratories I/O Stream Library to Standard C++ I/O Stream Library

The flag is still present in the MSVC library, as `ios_base::_Noreplace`, and in the Apache stdcxx implementation, as `ios_base::noreplace`.

Design Considerations

The historical name was "noreplace" but we could consider something like `ios_base::excl` to correspond more closely with POSIX and C. I originally preferred that, but have since decided that it's better to be consistent with the historical `noreplace` name, which is still present as `ios_base::_Noreplace` in MSVC. I think that the meaning of "noreplace" is a bit more intuitable than "exclusive". If you don't already know what POSIX or C means by "exclusive mode" then the name doesn't help you.

We could also consider not using an `ios_base::openmode` for this, but just add new constructors to `basic_filebuf`, `basic_ofstream` etc. to request the file be opened in exclusive mode. This would be novel, as all existing options for opening files (such as binary mode) are done via `openmode` flags. There was no interest in doing it any differently when the idea was discussed on the LEWG reflector.

Niall Douglas raised a concern related to the ISO C specification for `fopen`, which is vague about what "exclusive" mode means, and allows it to be ignored. I feel we should not deviate from C, so any fixes should be done "upstream" in the C standard. That makes it simpler to implement the C++ feature in terms of `fopen`, rather than having to do use OS-specific APIs.

Proposed wording

This is relative to the N4901 working draft.

Add a feature test macro to [version.syn]:

```
#define __cpp_lib_ios_noreplace   YYYYDDL // also in <ios>
```

Add a new `openmode` constant to the `ios_base` synopsis in [ios.base.general] as indicated:

```
// [ios.openmode], openmode
using openmode = T3;
static constexpr openmode app = unspecified;
static constexpr openmode ate = unspecified;
static constexpr openmode binary = unspecified;
static constexpr openmode in = unspecified;
static constexpr openmode out = unspecified;
static constexpr openmode trunc = unspecified;
static constexpr openmode noreplace = unspecified;
```

Add a new row to Table 123 [tag:ios.openmode]:

Element	Effect(s) if set
`app`	seek to end before each write
`ate`	open and seek to end immediately after opening
`binary`	perform input and output in binary mode (as opposed to text mode)
`in`	open for input
`out`	open for output
`trunc`	truncate an existing stream when opening
`noreplace`	open in exclusive mode

Add a new column to Table 130 [tab:filebuf.open.modes], as indicated:

`binary`	**`in`**	**`out`**	**`trunc`**	**`app`**	**`noreplace`**	
		+				"w"
		+			+	"wx"
		+		+		"w"

binary	in	out	trunc	app	noreplace	
		+	+		+	"wx"
		+		+		"a"
				+		"a"
	+					"r"
	+	+				"r+"
	+	+	+			"w+"
	+	+	+		+	"w+x"
	+	+		+		"a+"
	+			+		"a+"
+		+				"wb"
+		+			+	"wbx"
+		+	+			"wb"
+		+	+		+	"wbx"
+		+		+		"ab"
+				+		"ab"
+	+					"rb"
+	+	+				"r+b"
+	+	+	+			"w+b"
+	+	+	+		+	"w+bx"
+	+	+		+		"a+b"
+	+			+		"a+b"